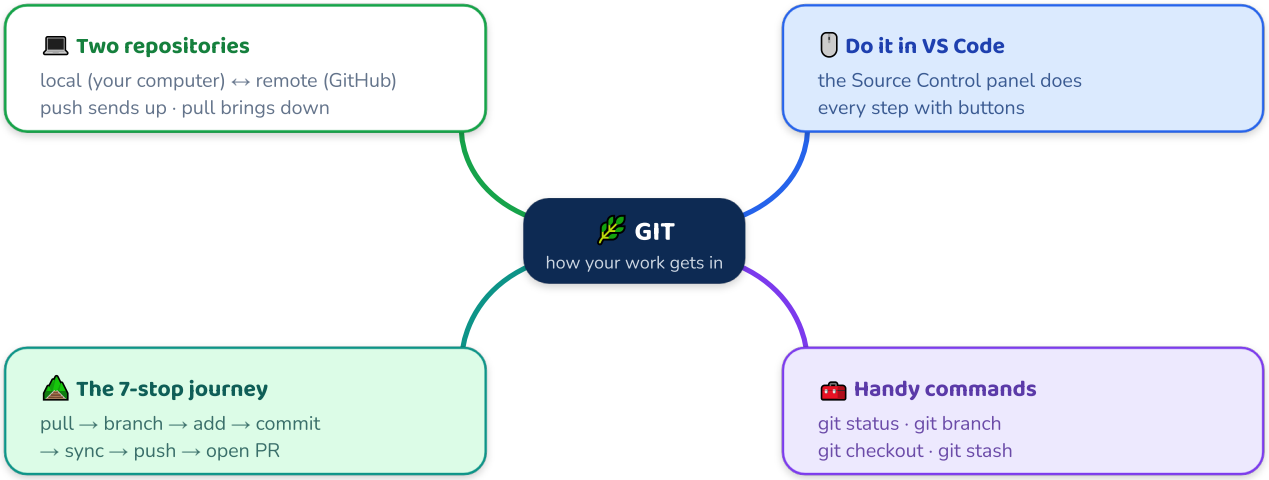
How to use Git — A Visual Guide

The everyday workflow: branch, commit, push, and open a pull request — explained simply.

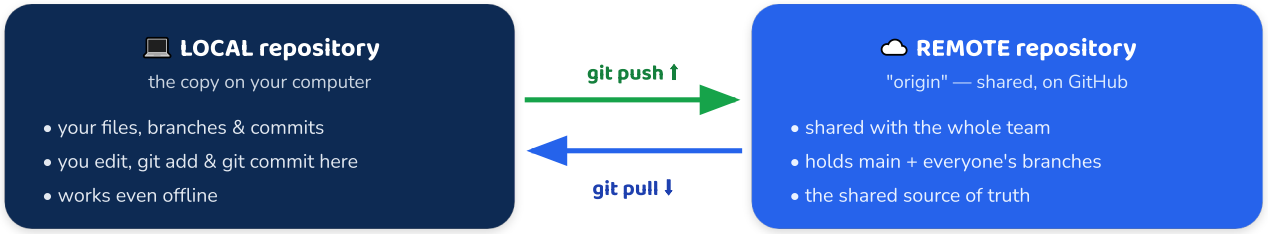
What's inside

Everything you need for day-to-day Git: the two repositories, the 7-stop journey, the VS Code way, and the handy commands.



1 · Git basics & the journey

Git keeps **two copies** of your project. You work on the **local** one on your computer; **push** and **pull** keep it in sync with the shared **remote** on GitHub.



push = send your commits up to the team · **pull** = bring everyone else's commits down

Golden rule: never work straight on **main**. The 7 stops below move your work from local up to the shared main.



Why a branch?

main stays clean and releasable. Your branch is a private sandbox — experiment freely, and nothing reaches main until it's reviewed.

The loop in one line

pull main → branch → add → commit → sync → push → PR → merge. Then start again for the next task.

2 · Stops 1–4, explained

Getting your work ready: start fresh → branch → stage → save.

1 Start from the latest main

```
git checkout main
git pull
```

`git checkout main` moves you onto the **main** branch, so the files you see are main's version. `git pull` then downloads everyone's latest commits from GitHub and merges them in — now you're standing on the newest shared code.

> Already on main? Skip the checkout and just run `git pull`. It's the short form of `git pull origin main` because main already **tracks** origin/main (more on that at stop 5).

2 Create your own branch

```
git checkout -b test/my-new-test
```

`-b` means *"make a new branch and switch to it"* in one step. A branch is your private copy of the code where you can change things without touching main. Name it for the work: `test/…`, `fix/…` or `docs/…`.

> Always branch off an **up-to-date** main — that's why stop 1 comes first.

3 Stage your changes — `git add`

```
git add .                              # stage everything you changed
git add tests/features/hotel.feature   # ...or one file at a time
```

Staging tells Git **which** changes go into the next commit. The `.` stages everything changed in the current folder and below. You can also add files one-by-one when you only want some of them in this commit.

> 🎨 **Colours** (in `git status` and VS Code): **red = changed but not staged yet**, **green = staged** (going into the commit). After `git add`, files flip from red to green.

4 Save it — `git commit`

```
git commit -m "test(sabre): add hold flight EWR-LHR price check"
```

`commit` saves your *staged* changes as a permanent snapshot in your branch's history. `-m` is the message that describes what you did.

> 📦 **add vs commit:** `git add` chooses *what* goes in (put items in the box). `git commit` *saves* the box with a label. Add stages; commit records — nothing is saved until you commit.

> ✍️ **What the message should contain:** `type(scope): summary` — e.g. `test(hotels): add refundable filter check`. **type** = test / fix / feat / docs / chore · **scope** = the area (sabre, hotels…) · **summary** = short, present-tense, says what the change does.

3 · Stops 5–7, explained

Sharing your work: sync → push → open the Pull Request.

5 Sync with main again

```
git pull origin main
```

Before pushing, pull main's latest **into your branch**. You're on your feature branch now, so this merges the team's newest main into your work — any conflicts appear here, on your machine, instead of in the PR.

❓ **Why not just `git pull`?** Plain `git pull` follows *your branch's* upstream (`origin/test/my-new-test`) — which has nothing new, since only you push there. To pull *main's* changes you must name it: `git pull origin main` = "from **origin**, the **main** branch, into where I am right now." That explicitness is the whole point.

6 Push your branch up

```
git push -u origin test/my-new-test    # the first time
git push                               # every time after
```

`push` uploads your commits to GitHub (the `origin` remote), onto a branch with the same name.

🔗 `git push origin test/my-new-test` pushes once. `git push -u origin test/my-new-test` pushes *and* links your local branch to the remote one (`-u` = set upstream). After that first `-u` push, plain `git push` and `git pull` just work. You only need `-u` the first time you push a new branch.

7 Open the PR — and where the link is

```
gh pr create --base main --fill
```

Open a Pull Request so a teammate reviews your branch before it joins main.

🔗 **Where you'll see the PR link:**

- Right after `git push`, the terminal prints a URL like `https://github.com/atriis/Atriis.Test.Automation/pull/new/test/my-new-test` — Ctrl-click it.
- On github.com, the repo shows a yellow **"Compare & pull request"** banner for your just-pushed branch.
- `gh pr create` prints the new PR's URL when it finishes.

🧹 **After it's merged:** `git checkout main` → `git pull` → `git branch -d test/my-new-test` to tidy up the branch you finished with.

4 · Other commands worth knowing

Not part of the main loop, but you'll reach for these often.

🔀 Switch to an existing branch — `git checkout <name>`

```
git checkout test/my-new-test    # jump onto a branch that already exists
git checkout main                # ...or back to main
```

Same `checkout`, but with **no** `-b` — because the branch already exists. `-b` *creates* a new branch; without it you just move onto one that's already there. (Newer Git also has `git switch <name>`, which does exactly the same thing.)

🌳 See your branches — `git branch`

```
git branch        # your local branches (* = the one you're on)
git branch -a     # include the remote branches too
git branch -d test/my-new-test   # delete a branch once it's merged
```

Lists your branches and marks the current one with `*`. Handy before switching, or to copy the exact branch name.

📦 Shelve work — `git stash` / `git stash -u`

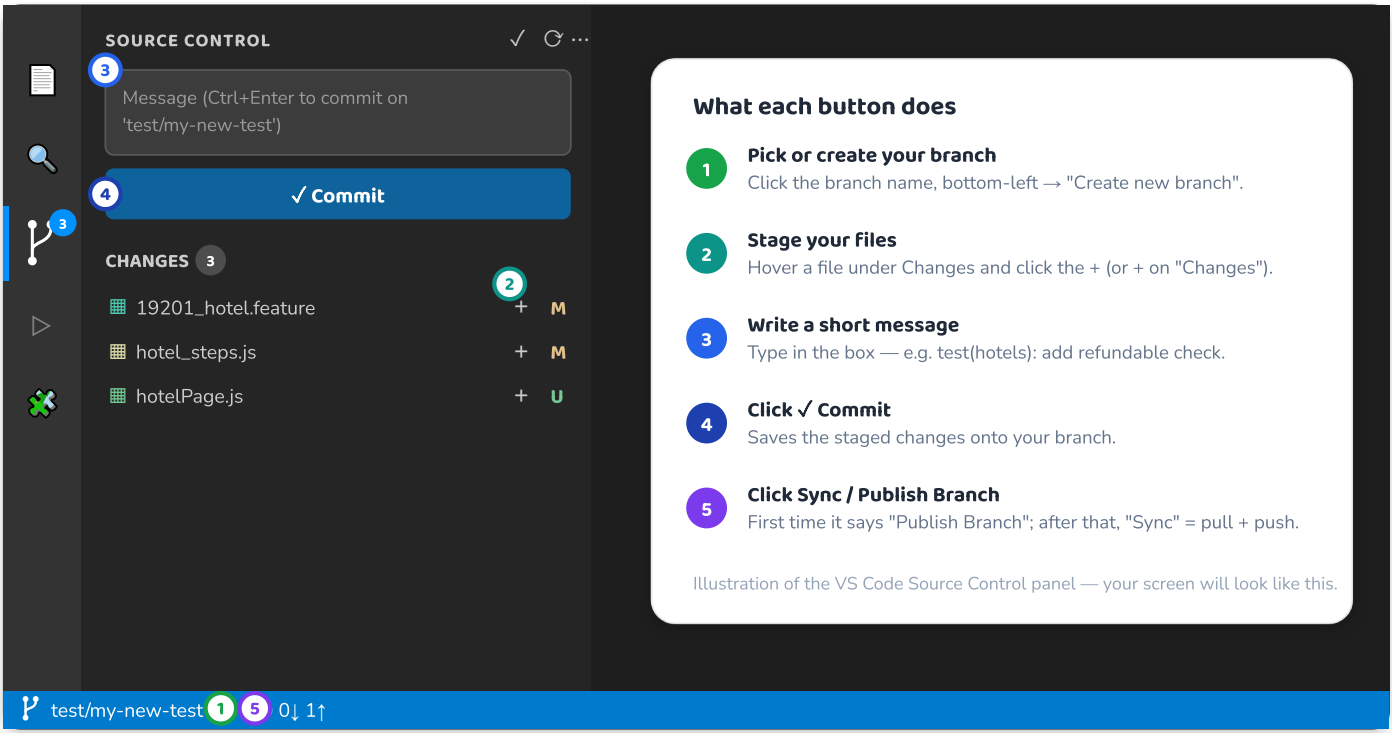
```
git stash        # shelve your tracked changes
git stash -u     # ALSO shelve new (untracked) files
git stash pop    # bring everything back
```

Parks your uncommitted changes so you get a clean tree — handy when you suddenly need to switch branch or pull but aren't ready to commit half-done work.

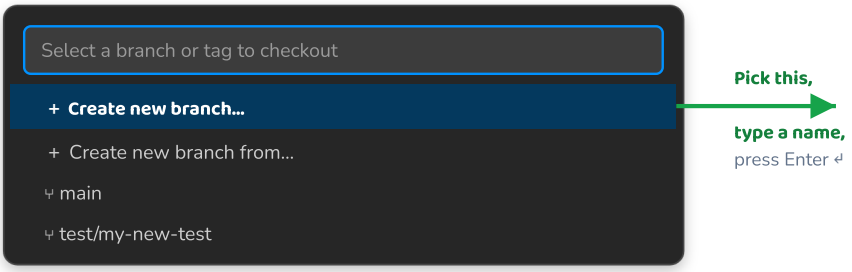
> `git stash` hides tracked (changed / staged) files but **leaves brand-new files behind**. `git stash -u` (*--include-untracked*) also sweeps up those new files. Either way, `git stash pop` brings them all back when you return.

5 · The same thing inside VS Code

Prefer buttons over commands? VS Code's **Source Control** panel does every step. Open it with the branch icon in the left bar (or **Ctrl+Shift+G**).



Stop 2 — making a branch: click the branch name in the bottom-left, and this menu pops up:



Even easier: GitHub Pull Requests extension

Install it to create and review PRs (stop 7) without leaving VS Code.

Reading the changes — colours & `git status`

Those little letters next to each file (and their colours) tell you what happened to it. The terminal says the same thing.

What the letters & colours mean

- **U Untracked** — a brand-new file Git doesn't know yet (green)
- **M Modified** — an existing file you changed (orange / gold)
- **A Added** — a new file you've staged (green)
- **D Deleted** — you removed the file (red)
- **C / ! Conflict** — needs you to resolve it (red)

`git status` — the terminal version

```
git status   # what changed + what's staged
```

It groups files into **Changes to be committed** (staged — shown **green**) and **not staged / Untracked** (shown **red**).

Quick rule: **green = ready to commit**, **red = still needs** `git add`.

6 · If something goes wrong

The everyday Git snags — and the quick way out of each.

Push rejected ("updates were rejected")

Someone pushed to main first. Run `git pull origin main`, fix any conflicts, then push again.

Merge conflict

Git marks the clashing spots with `<<<` / `===` / `>>>`. Edit the file to keep the right version, delete the markers, then `git add` & `git commit`.

Started work on the wrong branch

Nothing committed yet? Shelve it and move: `git stash -u` → `git checkout -b test/my-work` → `git stash pop`.

Typo in your last commit message

Before you push, fix it with `git commit --amend -m "test(scope): better message"`.

Not sure what state you're in?

`git status` shows your branch, what's staged (green) and what isn't (red). When in doubt, run it.

Pulled and got a scary "merge" message

Usually fine — Git just combined main into your branch. If there were conflicts, resolve them as above; otherwise you're good to push.